

Q1. CASE STUDY 1

CODE

```

#include <stdio.h>
#include <string.h>
// Union to store employment status memory-efficiently
// Only one status is active at a time, so union saves memory
union EmploymentStatus {
    char fullTime[15];
    char contract[15];
    char intern[15];
};
// Nested structure for project details
struct Project {
    char projectName[50];
    char role[30];
    int durationMonths;
};
// Main structure for employee, containing nested structure and union
struct Employee {
    int empID;
    char name[50];
    char department[30];
    char designation[30];
    float salary;
    struct Project project; // Nested structure
    union EmploymentStatus status; // Union for status
    int statusType; // 0 = Full-Time, 1 = Contract, 2 = Intern
};
// Global variable to maintain employee records across modules
struct Employee employeeRecords[100];
// Global count of employees, accessible across modules
int totalEmployees = 0;
// Static variable - retains value between function calls, limited to this file
static int internalCounter = 0;
// Function to add a new employee record (simulating a module)
void addEmployee(struct Employee emp) {
    employeeRecords[totalEmployees] = emp;
    totalEmployees++;
    internalCounter++; // static variable tracks internal operations
}
// Function to display employee details
void displayEmployee(struct Employee emp) {
    printf("\nEmployee ID : %d\n", emp.empID);
    printf("Name : %s\n", emp.name);
    printf("Department : %s\n", emp.department);
    printf("Designation : %s\n", emp.designation);
    printf("Salary : %.2f\n", emp.salary);
    printf("Project Name : %s\n", emp.project.projectName);
    printf("Role : %s\n", emp.project.role);
    printf("Duration : %d months\n", emp.project.durationMonths);
    printf("Employment Status : ");
    if (emp.statusType == 0)
        printf("%s\n", emp.status.fullTime);
    else if (emp.statusType == 1)
        printf("%s\n", emp.status.contract);
    else
        printf("%s\n", emp.status.intern);
}
int main() {
    struct Employee e1;
    // Filling employee details
    e1.empID = 101;
    strcpy(e1.name, "Rajesh Kumar");
    strcpy(e1.department, "Software Development");

```

```
strcpy(e1.designation, "Senior Developer");
e1.salary = 75000.50;
// Filling nested project structure
strcpy(e1.project.projectName, "Cloud HRMS");
strcpy(e1.project.role, "Backend Developer");
e1.project.durationMonths = 8;
// Setting employment status using union (only one is valid at a time)
e1.statusType = 0; // Full-Time
strcpy(e1.status.fullTime, "Full-Time");
// Add employee to global record array
addEmployee(e1);
// Adding another employee to demonstrate multiple records
struct Employee e2;
e2.empID = 102;
strcpy(e2.name, "Anjali Sharma");
strcpy(e2.department, "Data Analytics");
strcpy(e2.designation, "Intern");
e2.salary = 20000.00;
strcpy(e2.project.projectName, "Data Insights Dashboard");
strcpy(e2.project.role, "Data Analyst Intern");
e2.project.durationMonths = 3;
e2.statusType = 2;
strcpy(e2.status.intern, "Intern");
addEmployee(e2);
printf("=== Employee Records (Global Storage) ===\n");
for (int i = 0; i < totalEmployees; i++) {
    displayEmployee(employeeRecords[i]);
}
printf("\nTotal Employees Added: %d\n", totalEmployees);
printf("Static Internal Counter (module-level operations): %d\n", internalCounter);
return 0;
}
```

OUTPUT

(no output)

Q2. CASE STUDY 2

CODE

```

#include <stdio.h>
#include <string.h>
// Nested structure for tester details
struct Tester {
    int testerID;
    char testerName[50];
};
// Structure for developer details
struct Developer {
    int devID;
    char devName[50];
};
// Union for bug status - only one status active at a time
union Status {
    char open[10];
    char inProgress[15];
    char closed[10];
};
// Main structure for bug record
struct Bug {
    int bugID;
    char description[100];
    char severity[20];
    struct Developer dev;
    struct Tester tester; // nested structure
    union Status status;
    int statusCode;       // 1=Open, 2=In Progress, 3=Closed
};

void displayBug(struct Bug b) {
    printf("\n—— BUG REPORT ——\n");
    printf("Bug ID       : %d\n", b.bugID);
    printf("Description    : %s\n", b.description);
    printf("Severity       : %s\n", b.severity);
    printf("Developer ID   : %d\n", b.dev.devID);
    printf("Developer Name : %s\n", b.dev.devName);
    printf("Tester ID      : %d\n", b.tester.testerID);
    printf("Tester Name    : %s\n", b.tester.testerName);
    printf("Status         : ");
    if (b.statusCode == 1)
        printf("%s\n", b.status.open);
    else if (b.statusCode == 2)
        printf("%s\n", b.status.inProgress);
    else if (b.statusCode == 3)
        printf("%s\n", b.status.closed);
    printf("—————\n");
}

int main() {
    struct Bug bug1;
    int choice;
    bug1.bugID = 101;
    strcpy(bug1.description, "Login page crashes on invalid input");
    strcpy(bug1.severity, "High");
    bug1.dev.devID = 1;
    strcpy(bug1.dev.devName, "Rahul Sharma");
    bug1.tester.testerID = 501;
    strcpy(bug1.tester.testerName, "Priya Menon");
    printf("Select Bug Status:\n1. Open\n2. In Progress\n3. Closed\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    bug1.statusCode = choice;
    switch (choice) {
        case 1:

```

```
        strcpy(bug1.status.open, "Open");
        break;
    case 2:
        strcpy(bug1.status.inProgress, "In Progress");
        break;
    case 3:
        strcpy(bug1.status.closed, "Closed");
        break;
    default:
        printf("Invalid choice!\n");
        return 1;
}
displayBug(bug1);
return 0;
}
```

INPUT

1

OUTPUT

(no output)

Q3. CASE STUDY 3

CODE

```

#include <stdio.h>
#include <string.h>
// Nested structure for supplier details
struct Supplier {
    int supplierID;
    char supplierName[50];
    char contactNumber[15];
};
// Union to hold category-specific data (only one category applicable at a time)
union CategoryDetails {
    struct {
        char brand[30];
        int warrantyMonths;
    } electronics;
    struct {
        char size[10];
        char material[30];
    } clothing;
    struct {
        char expiryDate[15];
        char unit[10];
    } grocery;
};
// Enum to track which category is active in the union
enum ProductType { ELECTRONICS, CLOTHING, GROCERY };
// Main product structure with nested structure and union
struct Product {
    int productID;
    char productName[50];
    float price;
    int stockQuantity;
    enum ProductType type;           // Tells us which union member is valid
    union CategoryDetails details;   // Union for category-specific info
    struct Supplier supplier;        // Nested structure for supplier info
};
// Function to display product details based on active category
void displayProduct(struct Product p) {
    printf("\n-----\n");
    printf("Product ID      : %d\n", p.productID);
    printf("Product Name     : %s\n", p.productName);
    printf("Price              : %.2f\n", p.price);
    printf("Stock Quantity    : %d\n", p.stockQuantity);
    switch (p.type) {
        case ELECTRONICS:
            printf("Category          : Electronics\n");
            printf("Brand              : %s\n", p.details.electronics.brand);
            printf("Warranty           : %d months\n", p.details.electronics.warrantyMonths);
            break;
        case CLOTHING:
            printf("Category          : Clothing\n");
            printf("Size               : %s\n", p.details.clothing.size);
            printf("Material           : %s\n", p.details.clothing.material);
            break;
        case GROCERY:
            printf("Category          : Grocery\n");
            printf("Expiry Date        : %s\n", p.details.grocery.expiryDate);
            printf("Unit               : %s\n", p.details.grocery.unit);
            break;
    }
    printf("Supplier ID       : %d\n", p.supplier.supplierID);
    printf("Supplier Name     : %s\n", p.supplier.supplierName);
    printf("Contact Number    : %s\n", p.supplier.contactNumber);
}

```

```
printf("-----\n");
}
int main() {
    struct Product p1, p2, p3;
    // Product 1: Electronics
    p1.productID = 101;
    strcpy(p1.productName, "Smartphone");
    p1.price = 24999.00;
    p1.stockQuantity = 50;
    p1.type = ELECTRONICS;
    strcpy(p1.details.electronics.brand, "TechBrand");
    p1.details.electronics.warrantyMonths = 12;
    p1.supplier.supplierID = 1;
    strcpy(p1.supplier.supplierName, "Global Electronics Pvt Ltd");
    strcpy(p1.supplier.contactNumber, "9876543210");
    // Product 2: Clothing
    p2.productID = 102;
    strcpy(p2.productName, "T-Shirt");
    p2.price = 499.00;
    p2.stockQuantity = 200;
    p2.type = CLOTHING;
    strcpy(p2.details.clothing.size, "L");
    strcpy(p2.details.clothing.material, "Cotton");
    p2.supplier.supplierID = 2;
    strcpy(p2.supplier.supplierName, "Fashion World");
    strcpy(p2.supplier.contactNumber, "9123456780");
    // Product 3: Grocery
    p3.productID = 103;
    strcpy(p3.productName, "Rice Bag");
    p3.price = 850.00;
    p3.stockQuantity = 100;
    p3.type = GROCERY;
    strcpy(p3.details.grocery.expiryDate, "2026-05-01");
    strcpy(p3.details.grocery.unit, "5kg");
    p3.supplier.supplierID = 3;
    strcpy(p3.supplier.supplierName, "Farm Fresh Supplies");
    strcpy(p3.supplier.contactNumber, "9988776655");
    // Display all product details
    displayProduct(p1);
    displayProduct(p2);
    displayProduct(p3);
    return 0;
}
```

OUTPUT (no output)

Q4. CASE STUDY 4

CODE

```

#include <stdio.h>
#include <string.h>
typedef union {
    int code;                // stores 0/1/2
    char text[32];           // stores status string (optional)
} Status;
typedef struct {
    int day;
    int month;
    int year;
} Date;
typedef struct {
    char diagnosis[60];
    char doctorName[40];
    float treatmentCost;
    Date visitDate;
} Visit;
typedef struct {
    int patientId;
    char name[60];
    int age;
    char bloodGroup[4];
    char medicalHistory[200];
    Status status;           // union for memory optimization
    int numVisits;
    Visit visits[10];        // max 10 visits per patient (can be changed)
} Patient;
void printStatus(Status s, int statusCode) {
    // Using statusCode mapping (efficient with union's int member)
    const char *statusText = NULL;
    if (statusCode == 0) statusText = "Admitted";
    else if (statusCode == 1) statusText = "Discharged";
    else if (statusCode == 2) statusText = "Under Observation";
    else statusText = "Unknown";
    printf("%s\n", statusText);
}
int main() {
    Patient p;
    // Patient details
    p.patientId = 101;
    strcpy(p.name, "John Doe");
    p.age = 45;
    strcpy(p.bloodGroup, "O+");
    strcpy(p.medicalHistory, "Diabetes, Hypertension");
    // Status using union (store only integer code)
    // 0 = Admitted, 1 = Discharged, 2 = Under Observation
    p.status.code = 2;
    // Visits
    p.numVisits = 2;
    strcpy(p.visits[0].diagnosis, "Fever");
    strcpy(p.visits[0].doctorName, "Dr. Smith");
    p.visits[0].treatmentCost = 3500.50;
    p.visits[0].visitDate.day = 10;
    p.visits[0].visitDate.month = 6;
    p.visits[0].visitDate.year = 2026;
    strcpy(p.visits[1].diagnosis, "Viral infection");
    strcpy(p.visits[1].doctorName, "Dr. Brown");
    p.visits[1].treatmentCost = 2200.00;
    p.visits[1].visitDate.day = 14;
    p.visits[1].visitDate.month = 6;
    p.visits[1].visitDate.year = 2026;
    // Output

```

```
printf("Patient ID: %d\n", p.patientId);
printf("Name: %s\n", p.name);
printf("Age: %d\n", p.age);
printf("Blood Group: %s\n", p.bloodGroup);
printf("Medical History: %s\n", p.medicalHistory);
printf("Status: ");
printStats(p.status, p.status.code);
printf("\nVisits (%d):\n", p.numVisits);
for (int i = 0; i < p.numVisits; i++) {
    printf("\nVisit %d\n", i + 1);
    printf("Date: %02d/%02d/%04d\n",
           p.visits[i].visitDate.day,
           p.visits[i].visitDate.month,
           p.visits[i].visitDate.year);
    printf("Diagnosis: %s\n", p.visits[i].diagnosis);
    printf("Doctor: %s\n", p.visits[i].doctorName);
    printf("Treatment Cost: %.2f\n", p.visits[i].treatmentCost);
}
return 0;
}
```

OUTPUT

(no output)